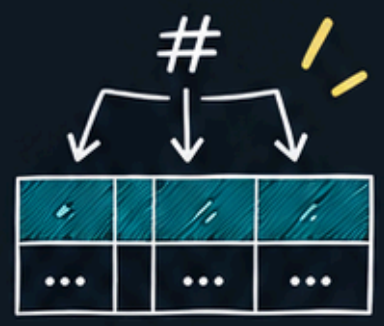
01 : ARRAYS & HASHING



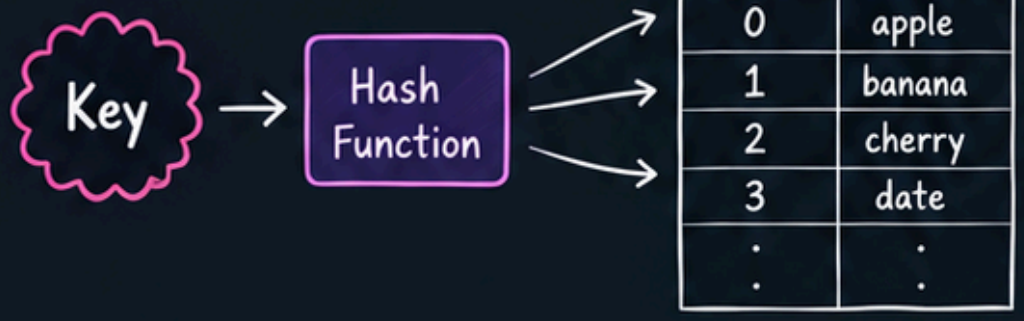
The foundation of most coding problems

1 Hash Map / Hash Set

Use when you need:

- Fast lookup
- Frequency counting
- Duplicate detection
- Grouping elements

Typical: O(n) time



2 Prefix Sum

Use when problems involve:

- Range sums
- Subarray sums
- Repeated sum queries

Core idea:

```
prefix[i] = prefix[i-1] + nums[i]
```

Index	0	1	2	3	4
nums	3	1	2	5	4
prefix	3	4	6	11	15

3 Kadane's Algorithm

Use for:

- Maximum subarray sum
- Best contiguous segment

Core idea:

```
current = max(num, current + num)
```

nums	-2	1	-3	4	-1	2	1	-5	4
current	-2	1	-2	4	3	5	6	1	5

max sum = 6

4 Frequency Counting

Think HashMap / Counter when you see:

- Anagrams
- Character frequency
- Most/least frequent elements
- Duplicate values

Example:

nums = [4, 1, 2, 4, 2, 4, 3]

Element	1	2	3	4
Count	1	2	1	3

PATTERN SIGNAL

"LOOKUP • COUNT • DUPLICATE • SUBARRAY"

→ THINK ARRAYS + HASHING

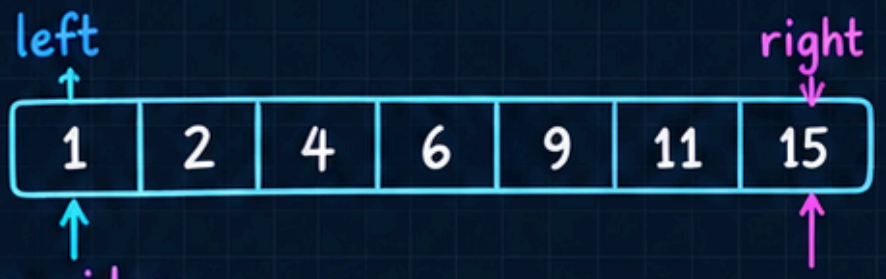
02: TWO POINTERS & SLIDING WINDOW

Turn repeated scanning into a single efficient pass.

1 Two Pointers

Use when you see:

- Sorted array
- Pair / triplet problems
- Compare elements from both ends
- In-place array manipulation



Core idea:

left → ← right

Move pointers based on the required condition.

Typical: O(n)

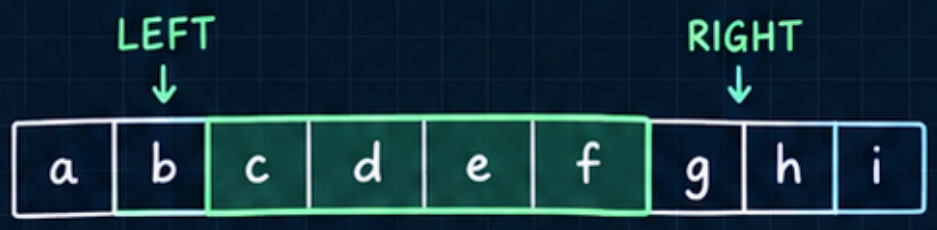
2 Sliding Window

Use when you see:

- Contiguous subarray / substring
- Longest or shortest segment
- Maximum / minimum within a range
- A condition that changes as the window moves

Core idea:

Expand → Check → Shrink



Typical: O(n)

3 Kadane's Algorithm

Use for:

- Maximum subarray sum
- Best contiguous segment

Core idea:

current = max(num, current + num)

PATTERN SIGNAL

★ "LOOKUP • COUNT • DUPLICATE • SUBARRAY"

→ THINK ARRAYS + HASHING

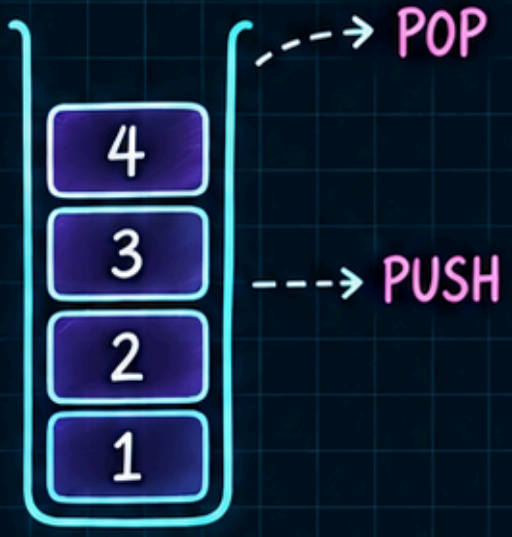
03 STACK & MONOTONIC STACK

Think **Stack** when the problem involves order, nesting, or the nearest element.

1 Stack

Use for:

- Matching parentheses
- Valid brackets
- Undo / backtracking
- Expression evaluation
- Nested structures



Core idea:

PUSH → PUSH → POP → POP

Typical: O(n)

2 Monotonic Stack

Use when you see:

- Next Greater Element
- Next Smaller Element
- Previous greater / smaller
- Daily temperatures
- Stock span
- Largest rectangle

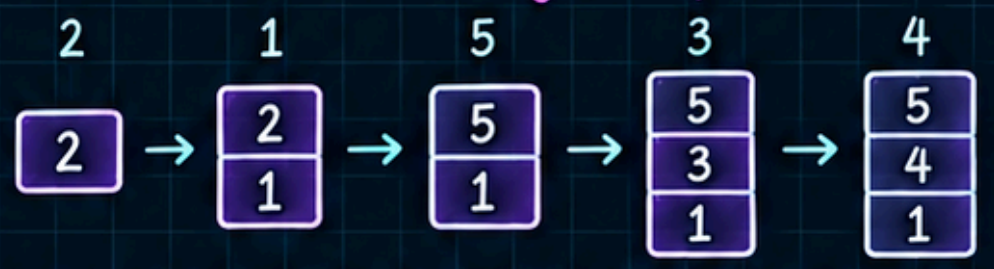
Core idea:

Maintain elements in **increasing** or **decreasing** order.

Each element is pushed and popped at most once.

Typical: O(n)

Monotonic Increasing (example)



Pattern Signal

★ "NESTED / MATCHING" → STACK

★ "NEXT / PREVIOUS GREATER OR SMALLER" → MONOTONIC STACK

04. BINARY SEARCH & INTERVALS

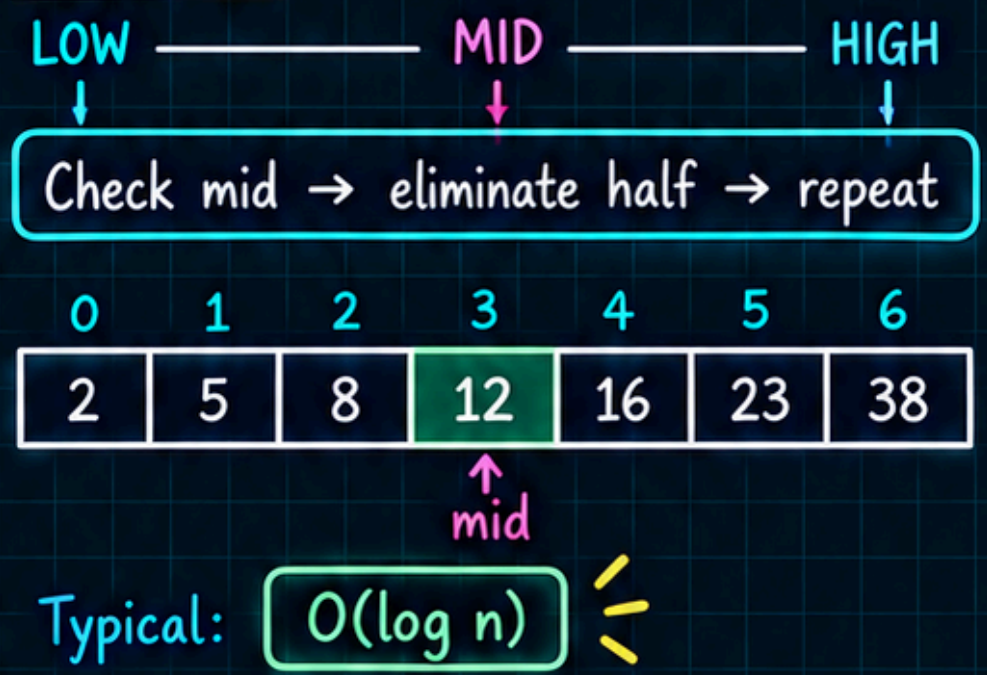
Reduce the search space or handle overlapping ranges efficiently.

1 Binary Search

Use when you see:

- Sorted array
- Search for a target
- First / last occurrence
- Minimum or maximum possible value
- Search on answer

Core idea:



Typical: O(log n)

2 Intervals

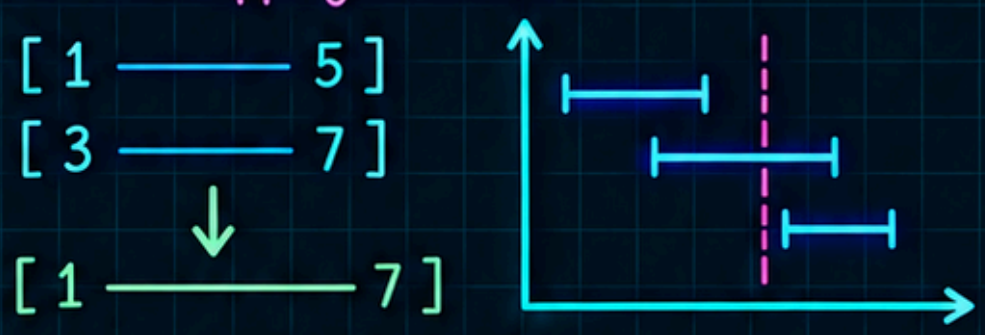
Use when you see:

- Overlapping ranges
- Meeting schedules
- Start / end times
- Merge intervals
- Minimum rooms required

Core idea:

Sort → Compare → Merge

For overlapping intervals:



Typical: O(n log n) due to sorting

Pattern Signal

★ "SORTED / SEARCH SPACE" → BINARY SEARCH

★ "START + END / OVERLAPPING RANGES" → INTERVALS

05. LINKED LIST

Think pointers, connections, and controlled movement.

1 Reverse Linked List

Use when you need to:

- Reverse a list
- Reverse part of a list
- Reconnect nodes

Core idea:

PREV ← CURRENT → NEXT

Move forward while reversing each pointer.

Typical: O(n) time • O(1) space

2 Fast & Slow Pointers

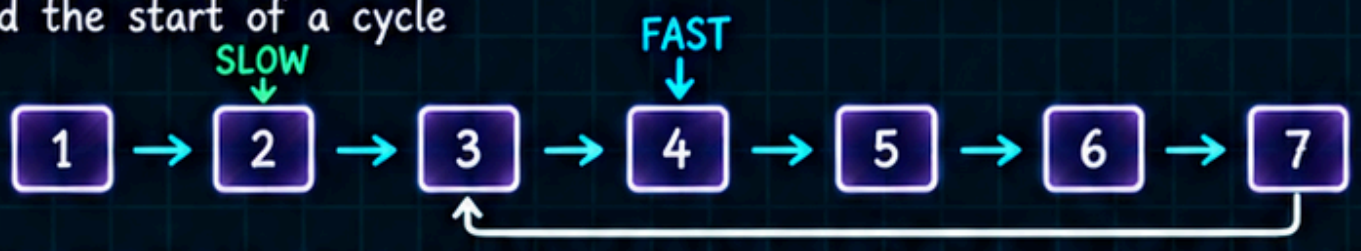
Use for:

- Cycle detection
- Find the middle node
- Detect circular structures
- Find the start of a cycle

Core idea:

SLOW → 1 step
FAST → 2 steps

If they meet → Cycle exists

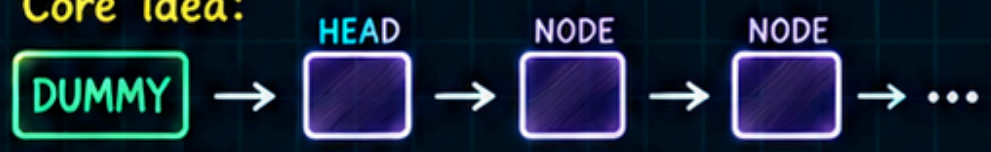


3 Dummy Node

Use when:

- Inserting/removing nodes
- Merging linked lists
- The head might change

Core idea:



It simplifies edge cases involving the first node.

Pattern Signal

- ★ "LINKED NODES + REVERSE" → REVERSE POINTERS
- ★ "CYCLE / MIDDLE" → FAST & SLOW
- ★ "HEAD MAY CHANGE" → DUMMY NODE

06. TREES & GRAPHS

Think traversal, connectivity, paths, and dependencies.

1 Tree Traversal

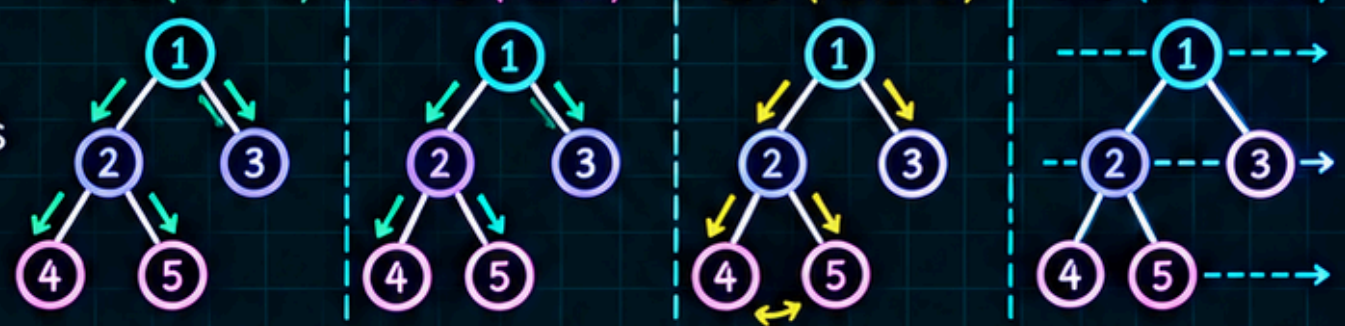
Use for:

- Tree search
- Height / depth
- Path problems
- Subtree calculations

Core patterns:

DFS → Preorder • Inorder • Postorder

BFS → Level-order traversal



2 Graph DFS / BFS

Use for:

- Connected components
- Islands / grids
- Reachability
- Shortest path in an unweighted graph

Core idea:

START → VISIT → EXPLORE → REPEAT

Typical:

O(V + E)

3 Topological Sort

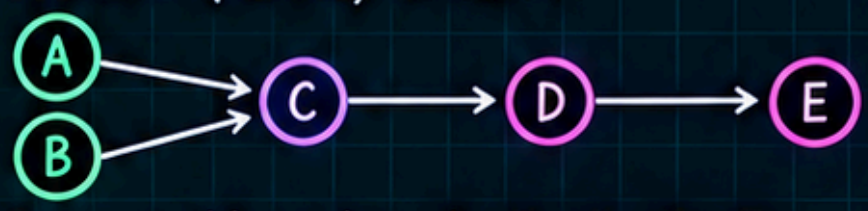
Use when you see:

- Dependencies
- Prerequisites
- Task ordering
- Course scheduling
- Directed acyclic graphs

Core idea:

DEPENDENCY → TASK → NEXT TASK

Use BFS (Kahn's) or DFS.



One possible order: A, B, C, D, E

Pattern Signal

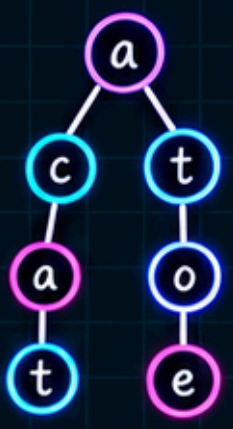
★ "TREE / HIERARCHY" → DFS / BFS

★ "CONNECTED / REACHABLE" → GRAPH DFS / BFS

★ "DEPENDENCY / PREREQUISITE" → TOPOLOGICAL SORT

07. HEAP, TOP K & TRIE

Choose these when you need priority, ranking, or prefix-based search.



1 Heap / Priority Queue

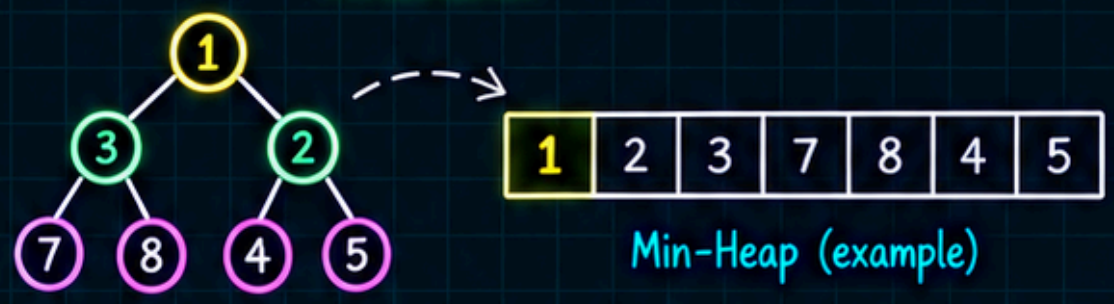
Use for:

- Top K elements
- Kth largest / smallest
- Scheduling
- Running median
- Merge K sorted lists

Core idea:

INSERT → PRIORITIZE → POP

Typical: O(log n) per operation



2 Top K Pattern

Use when the problem asks for:

- K largest / smallest
- K most frequent
- K closest elements
- Best K candidates

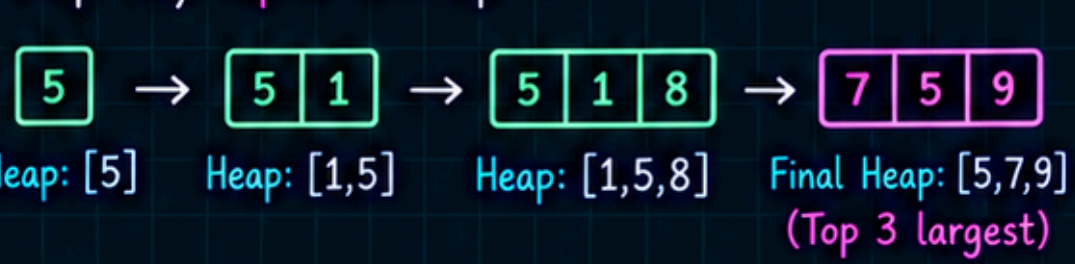
Example:
Find 3 largest numbers
[5, 1, 8, 3, 7, 2, 9]

Typical: O(n log k)

Core idea:

Maintain a heap of size K.

Process each number, keep only top 3 in heap.



3 Trie

Use for:

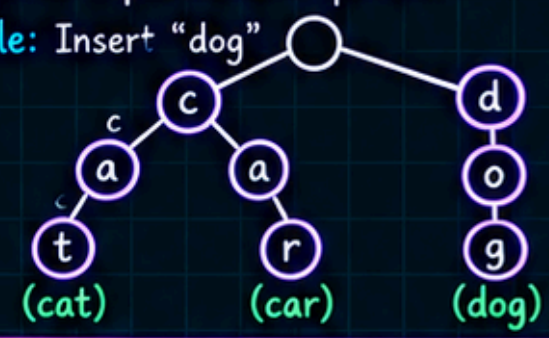
- Prefix search
- Autocomplete
- Dictionary lookup
- Word search
- Prefix matching

Core idea:

WORD → CHARACTER → CHARACTER → CHARACTER

Each path represents a prefix.

Example: Insert "dog"



Search "ca"
Prefix exists ✓

Pattern Signal

- ★ "TOP K / KTH / PRIORITY" → HEAP
- ★ "PREFIX / WORD DICTIONARY" → TRIE

08. BACKTRACKING & GREEDY

Explore all valid possibilities or make the best local choice.

1 Backtracking

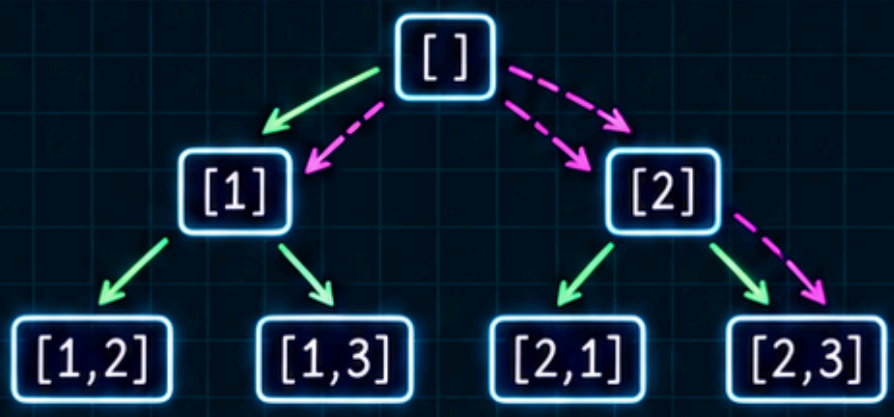
Use for:

- Generate all possibilities
- Permutations
- Combinations
- Subsets
- N-Queens
- Sudoku

Core idea:

CHOOSE → EXPLORE → UNCHOOSE

Build a solution step by step and undo the choice when needed.



Typical:

$O(2^n)$ to $O(n!)$

2 Greedy

Use when:

- A local best choice can lead to a global optimum
- Scheduling problems
- Activity selection
- Minimum number of resources
- Interval optimization

Core idea:

CHOOSE BEST → COMMIT → MOVE FORWARD

No backtracking after a choice.

Activities (sorted by end time)

[1,3]	[2,4]	[3,5]	[0,6]	[5,7]	[6,8]	[8,9]

Greedy picks

[1,3] ✓ → [3,5] ✓ → [5,7] ✓ → [8,9] ✓

Typical:

$O(n \log n)$ (due to sorting)

Pattern Signal

★ "GENERATE ALL / EVERY POSSIBILITY" → BACKTRACKING

★ "BEST CHOICE NOW / OPTIMIZE" → GREEDY

09. DYNAMIC PROGRAMMING

Use DP when a problem has overlapping subproblems and optimal substructure.

1 Knapsack DP

- **0/1 Knapsack**
  Each item → Take OR Skip
- **Unbounded Knapsack**
  Each item → Take Multiple Times
- **Pattern:**

`dp[state] → best possible answer`

0/1 Knapsack (Example)

Item	Weight	Value
1	2	3
2	3	4
3	4	5

Unbounded Knapsack (Example)

Item	Weight	Value
1	2	3
2	3	4
3	4	5

2 Sequence DP

- **LCS** → Longest Common Subsequence
- **LIS** → Longest Increasing Subsequence
- **Edit Distance** → Minimum operations to transform one string

Think:

`State → Transition → Base Case`

Examples

LCS X = A B C D A B, Y = B D C A B A — LCS Length = 4

LIS A = 10, 9, 2, 5, 3, 7, 101, 18 — LIS Length = 4 (2, 3, 7, 101)

Edit Distance word1 = kitten, word2 = sitting — Edit Distance = 3

3 Grid DP

Use for:

- Number of paths
- Minimum / maximum path cost
- Obstacles
- Grid-based optimization

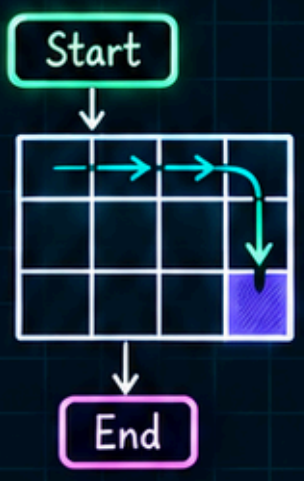
Core idea:

`dp[i][j] ← TOP + LEFT`

Example (Unique Paths)

	0	1	2	3	4
0	1	1	1	1	1
1	1	2	3	4	5
2	1	3	6	10	15
3	1	4	10	20	35

dp[i][j] = number of ways to reach (i, j)



Pattern Signal

- ★ "CHOICES + OVERLAPPING SUBPROBLEMS" → DP
- ★ "SEQUENCE / STRING" → LCS · LIS · EDIT DISTANCE
- ★ "GRID + PATH / COST" → GRID DP

10. PATTERN RECOGNITION

Don't memorize solutions. Recognize the structure.

LOOK FOR THESE SIGNALS

SORTED ARRAY	→ Binary Search / Two Pointers
CONTIGUOUS SUBARRAY / SUBSTRING	→ Sliding Window / Prefix Sum / Kadane
PAIR / TRIPLET	→ Two Pointers / Hashing
OVERLAPPING RANGES	→ Intervals
NEXT / PREVIOUS GREATER OR SMALLER	→ Monotonic Stack
TREE / CONNECTIVITY / PATH	→ DFS / BFS
TOP K / KTH / PRIORITY	→ Heap
PREFIX / AUTOCOMPLETE	→ Trie
GENERATE ALL POSSIBILITIES	→ Backtracking
DEPENDENCIES / PREREQUISITES	→ Topological Sort
OVERLAPPING SUBPROBLEMS	→ Dynamic Programming

THE 10-SECOND RULE

10 SEC

Read the problem → Identify the signal → Pick the pattern → Then code.